COURSE CODE: CSA1412

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: ***Construct top-down and bottom*** *up parsers* using *CFG for parsing* conflicts *and* error *recovery* ***for efficient syntax*** **tree** ***generation***. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: 3

Duration :60 Mins

Off **Class**

Total **Marks**:**30**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

1 Mathers & P–192311363 *Name / Reg No) certify that this submission is my original work and that* I hat *adhered to the guidelines specified for this assessment*.
I understand that any *violation of academic integrity rules will* result *in disciplinary* action.

Signature ***of the*** *student:*

P.Mathusi

CSAIL, 12- Compiler Design

CO2-AT3 Scenario Based Task)

Scenario 1 : context free Grammer CCFG)

(@) construct a context.

free grammer (CFG)

San-

Let the start symbol be E

E-TE'

E ' → +TE' /ε

TFT'

F→ (E)/id

T'→→ *FT'(ε

This

Mathush P 192311363

b) Explain how the

grammar ensures operator precedence and nested expression

The grammar ensures operator procedence by seperating expressions into different levels

E→ TE' handles addition C+)

TFT' handles

multiplication (*)

Since multiplication is processed inside T before addition in E, the multiplication operator has higher precedence than addition

For example: id tid id

*

is interpreted as: id + Cid kid)

The

production: FCE)

allows expresions inside parentheses. Therefore, nected expression such as such as cid+id) id are are also accepted.

(0) Determine whether the grammer is ambiguous unambiguous justif you answer.

The given grammer is

unambiguous

Cuiliferation

Jus

The

COD

grammer clearly definer operator precedence

Multiplication is evaluated before addition Parantheres determine the order of evaluation

*E

Every valid expression produces only one

unique parse tree

(d) show the left mort derivation for the expression:

id +id *id

Stack

input string

Action

$

id *id+id $

no action

$id

*id+id $

reduce with E→id.

$id

*id+id$

reduce with

E->id

$E

*id +id$

shift

$E*

id +id $

shiff,

$ E* id

+ id $

reduce with

Eid

$E*E

+id$

reduce with E→ E*E

Be

+id $

shift

$E+

id$

shiff

BE+id

$

$E+E

$1

$E

$

reduce with

E-id

reduce with

E→E+E

Accept

2

(e) Draw the Parse tree id id tid.

LU

E

E

+

E

-> id id +id/

E

E

id

id

(f) Why is this

- The

grammar suitable for Predictive parring? given grammer is suitable

for predictive parsing because:

it does not contain *left* recursion it is left factored

it satifies the LLCI) properly

only one look

ahead rymbal is required to `select` the correct *production*. `Therefore,` it can be parsed efficiently cuing

a predictive parser

3

2

(a) Explain the concept of shift reduce parsing and how it works in bottom-up parsing

The shift reduce parsing is a bottom up parsing technique that constructs the parse tree from input.

string to the start cymbal.

лутбы

it performs fore operations..

shift more the next input symbal to the stack

→C shift

ave

Reduce Replace the handle on the stack with its non-terminal.

→ Accept inpect string is valid.

c

->> © Emor input is invalid

(b) Identify the handles in input string 0011:

grammar： S→OSI /01

input：0011

Handles are reduced in the following order:

101 2)051

These handles one reduced until the start symbol

s is obtained

(c)

Perform shiff reduce parsing.

stack

input

Action

$

0011$

shift

$0

011$

shif

$00

11 $

shift

$001

1$

Redure 01-S

19

shift

$051

$

reduce 051-5

$5

$

Accepted

a) Perform shift reduce

parsing . & constreet rightmort

Derivation in Reverse

(Reverse) (Right mext) (Derivation

0011

→051

175

Rigt most Derivation

S.

051

<-001

4

(F)

@

Determine whether the

* *The*

Justification:-

grammer is ambiguous cars unambiguous. grammer is unambiguous.

Every valid etring has only one derivation

* Every valid ebring has only one pare

i

tree

* Therefore, the

cgramme à unambiguous

suggest how the parrer detects errors.

Example : *0101*

→ *The*

*

parser detects an error when

No valid

production nule

can be applied

The `input` connat

be reduced to `the`

start Symbol 3.

The stack and input do not match the

grammcu.

Determine whether the

*The goarmer датта

Justification:-

is

grammer is

ambiguou

is ambiguous cars

unambigues,

* Every valid string has only one derivation

* valid ebring has only one parse tree

Every * Therefore, the cgramme à unambiguous is

suggest how the parser detects errors.

Example : *0101*

→→ The parser detects an error when

No valid production nule can be applied

The

The input connat be reduced to the start Symbol 3.

grammou

The stack and input do not match the

Example lum.)

E→E+E

- E→ E*E

-E-id

/p string id tid & id

Stack

i/p String

Action .

$

id + id * id $

shift

$id

+id *id $

reduce E-id

Be

tid *id $

shiff

$E+

id kid $

shift

$ E+id

* id $

reduce E-id

BE+E

* id $

reduce with E~ETE

$E

*id $

$E*

id$

$ Eid

$

FA

$E

$

shift

Shift reduce E-id

reduce wilt E-ED

Accepted

Bottom

ир

Parse free

*

id

+

E

E

id